V1 vs V2

The same app, rebuilt around one idea.

V1's unit of work is a function call. V2's unit of work is an object. That is the whole change. A function call cannot be named, cancelled, resumed, replayed or attributed after the fact — which is why V1 has no stop button, loses a reply on refresh, and cannot tell you which answer a token belongs to. Everything in the table follows from it.

Every V1 claim below was checked against the code in **backend/**, not read off the architecture document.

		V1	V2	Checked against
1	**The unit of work**	A function call. **core.handle_text_input(text)** runs the whole pipeline and returns nothing. A function call has no name, so nothing downstream can refer to it.	An object. A **Turn** has an id, a status, a sequence number and a row in a table. Everything below is a consequence of this one difference.	`backend/core.py:336 _process_input_locked(...)` `v2/.../chat/turns.py Turn`
2	**Sending a message**	**POST /message** hands the text to a background task and replies **{"status": "ok"}**. You are not told what you started.	The POST returns a **turn_id**. That id is what makes every other row on this page possible.	`backend/server.py:253 return {"status": "ok"}`
3	**Stopping a reply**	You cannot. There is no stop, cancel or abort endpoint anywhere in server.py — not hidden, not broken: absent. Nothing has an id to cancel.	**POST /turns/{id}/cancel**, and it is idempotent — cancelling an already-finished turn succeeds rather than erroring.	`grep -i 'stop|cancel|abort' backend/server.py -> 0 routes` `v2/.../app.py:374 cancel_turn`
4	**Which reply is this token part of?**	Unanswerable. A token event carries **{"text": ...}** and nothing else — no turn, no conversation, no sequence. The client appends to one global buffer and hopes.	Every event carries **conversation_id**, **turn_id** and a sequence. Two turns can stream at once without either corrupting the other.	`backend/core.py:586 broadcast('token', {'text': ...})` `v2/.../kernel/events.py:88 emit()`
5	**Refreshing mid-reply**	The reply is lost. There is nothing to resume from — the tokens were broadcast and never written down as events.	The client holds a cursor and asks for everything after it. Events are an append-only log, so a reconnect replays exactly what was missed.	`v2/.../app.py:180 for event in bus.replay(last_seen, ...)`
6	**When the provider errors**	It is broadcast as a chat **message** from a sender called **Primnox**. A provider failure is rendered as though the assistant said it — which is how 'error thinking: Expecting value' became a reply.	A failure is **turn.failed**, with a code, a message and a retryable flag. It renders as an error with a retry button, never as an answer.	`backend/core.py:526 broadcast('message', {'sender': 'Primnox', 'text': _explain_...})`

7	**The event vocabulary**	Whatever string was typed at the call site. There is no registry, so a typo is a silently unhandled event.	37 registered kinds. Emitting an unregistered one raises immediately, at the emit, rather than going quiet on the wire.	`if kind not in ALL_KINDS:`
				`  raise ValueError(...)`
				`37 = 28 conv + 2 system + 7 ambient`

8	**Two messages at once**	A per-session **threading.Lock** held across the entire pipeline — routing, memory search, context assembly, the model call, persistence. The second message waits for all of it.	Turns are queued work with their own states. Concurrency is a scheduler concern, not a lock wrapped around the world.	`backend/core.py:42` `_session_locks_guard = Lock()`

9	**What state the app is in**	One global scalar, broadcast to everyone: **state = idle**. With two conversations open it describes neither.	Nine states, per turn, with legal transitions declared. **thinking** and **streaming** are separate on purpose — one is waiting on the provider, the other is receiving, and one spinner cannot say which.	`backend/server.py:2245`
				`broadcast('state', {'value':'idle'})`
				`v2/.../chat/turns.py:28 transitions`

10	**Where state lives**	The frontend reconstructs it from a stream of anonymous broadcasts. If the UI needs to know something, it infers it.	The backend owns state, the frontend renders it. The client never guesses — if the UI knows something, the backend said it.	`docs/ARCHITECTURE_V2.md §1.1`
11	**Storage**	**chat.db** + **memory.db** + **settings.json** + a vault file + rotating backups. State and the thing describing it commit separately.	One **primnox.db**. An event and the state change it describes commit in the same transaction or not at all.	`backend/backup_manager.py:208` `v2/README.md storage table`
12	**Backend size**	37,584 lines across 146 Python files. **server.py** alone is 2,844 and **core.py** 952.	27,283 lines across 123 files, in 16 packages with one owner each. **app.py** is 845.	`find -name '*.py' | xargs wc -l`
13	**Frontend size**	14,916 lines. Four components are over 1,000 lines each — CalendarView is 1,334.	4,613 lines. The largest file is App.tsx at 901, and it is the shell.	`1334 CalendarView.tsx (V1)` ` 901 App.tsx (V2)`
14	**Tools**	A loop inside **brain.py**, with provider quirks handled by **_rejects_tools**-style special cases in the same file.	A registry of 14 specs with declared danger levels, and a runtime that is separate from any provider.	`backend/brain.py (1,794 lines)` `v2/.../tools/ 14 registered specs`
15	**Building the prompt**	Assembled inline in the pipeline. Whatever fits, fits.	A context service with an explicit token budget: retrieval first, then assets, then history — and history stops at the first turn that does not fit rather than leaving a hole in the middle.	`v2/.../context/service.py:211` `if spent + cost > budget: break`

16	**Knowledge**	Notes and memories in a table, searched by keyword.	A knowledge graph over indexed code and documents — 2,782 nodes, 3,913 edges — returning file:line citations instead of pasted source.	`v2/.../knowledge/graph.py` `knowledge_nodes / knowledge_edges`
17	**Configuration**	Constants in source. Changing one means editing Python.	27 tunables with min, max, type, a plain-English summary and the cost of getting it wrong — over an API, with a UI, and an env var outranks both.	`v2/.../settings/tunables.py` `_t('context.history_turns', 2000, ...)`
18	**Tests**	1,070 test functions across 67 files. The larger number, and worth saying plainly — V1 is not untested.	317 across 30 files, but layered on purpose: L0 contracts, L1 unit, L2 integration and HTTP, L3 scenarios, L4 chaos, plus golden conversations and performance budgets. 535 assertions pass in 63 seconds.	`grep -c '^def test_'` `V1 1070 / V2 317`
19	**What it can do**	More. Calendar, Notes, Meetings, Research, Onboarding, a command centre and a dynamic island — none of which V2 has.	Less, on purpose. Chat, assets, workspaces, knowledge, memory, skills and settings, each on a contract that the missing features can be built on later.	`frontend/src/app/components/` `27 views in V1`

What that adds up to

The honest summary	V2 is not a bigger V1.	It is a smaller one with a spine. 10,000 fewer backend lines and 10,000 fewer frontend lines, doing fewer things — but every one of them addressable, cancellable and replayable. V1 does more and can prove less.
What V1 got right	Carried over rather than rebuilt.	Loopback-only binding with the WebSocket origin allowlist (subtle and correct — V2 shipped it with a hole and had to fix it). The Privacy Mirror as a mechanism. Database corruption recovery. Context-overflow handling. The local-provider circuit breaker. And **_explain_api_error**, which in V2 fills turn.failed.message instead of faking a reply.
What is still V1's	V2 runs alongside it, not instead of it.	Different ports, different database — **:4009** and **:4109**, **chat.db** and **primnox.db**. Nothing in v2/ can break the shipping app. The Privacy Gateway exists in V2 as a boundary only; the scrub-and-rehydrate mechanism is still V1's.

One row is worth reading twice. Row 18: V1 has three times as many test functions as V2. The number that matters is not which is larger but what each is testing — V1's cover behaviour that a rewrite is free to change, V2's cover a contract that it is not.